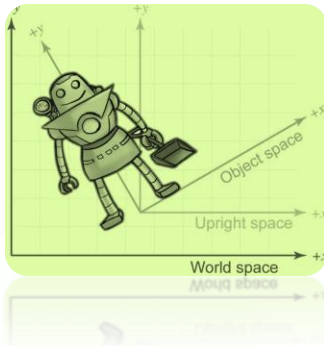


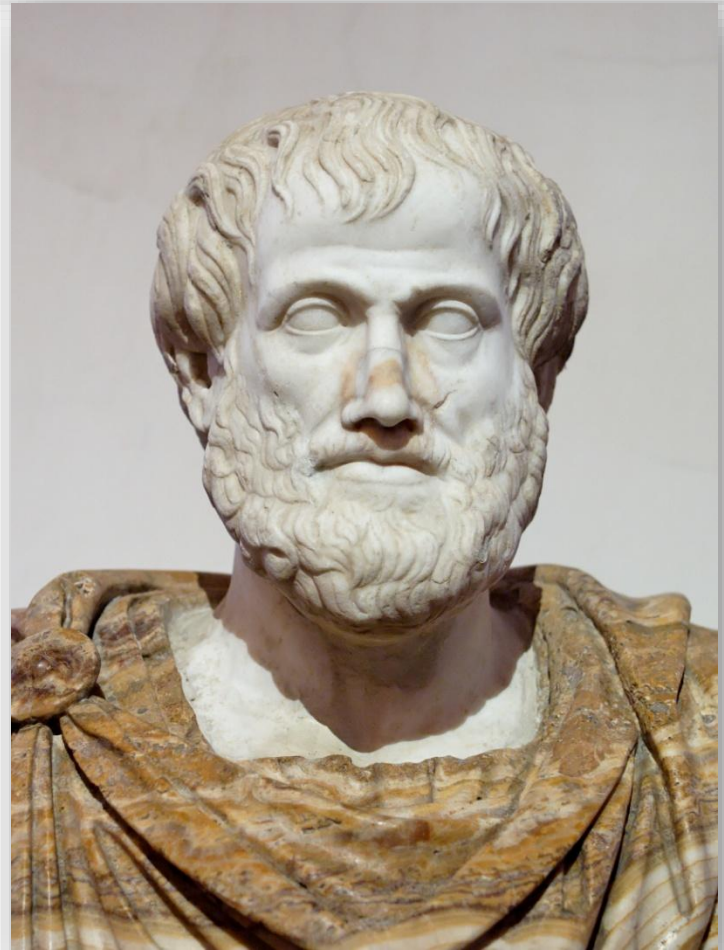
Chapter 3: Multiple Coordinate Spaces

Fletcher Dunn
Valve Software

Ian Parberry
University of North Texas

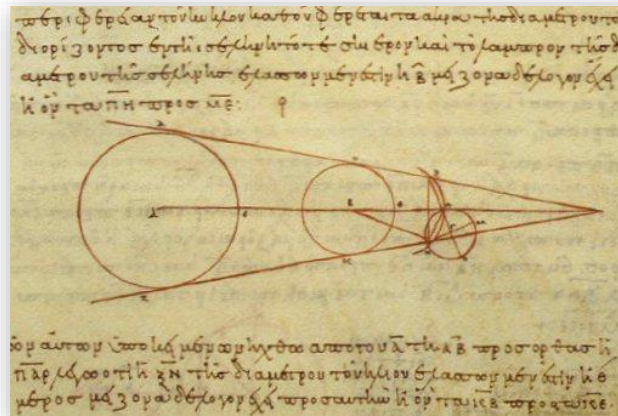
Aristotle

- Aristotle (384-322 BCE) proposed a geocentric universe with the Earth at the origin.
- (Image from Wikimedia Commons)



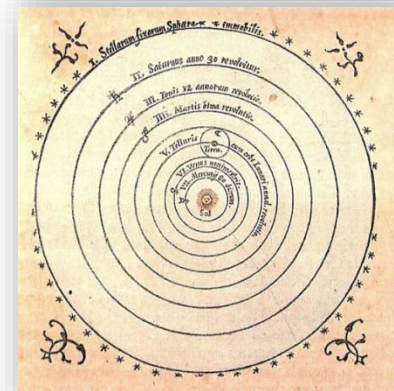
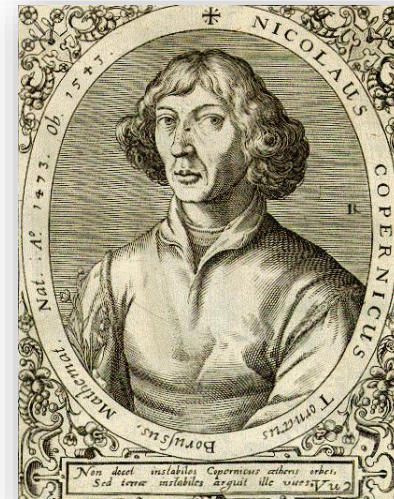
Aristarchus

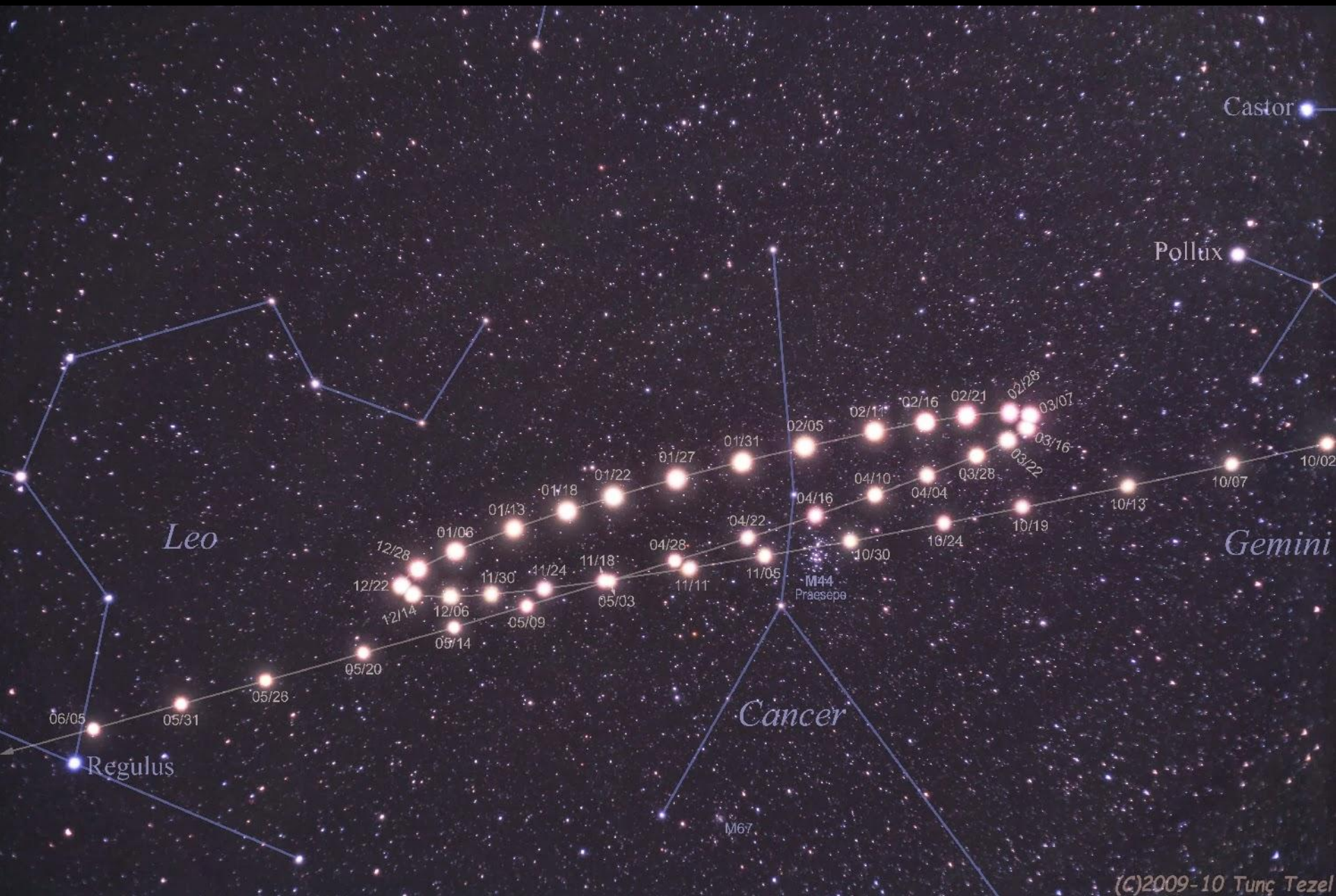
- Aristarchus (ca. 310-230 BCE) proposed a heliocentric universe with the Sun at the origin.
- (Images from Wikimedia Commons)

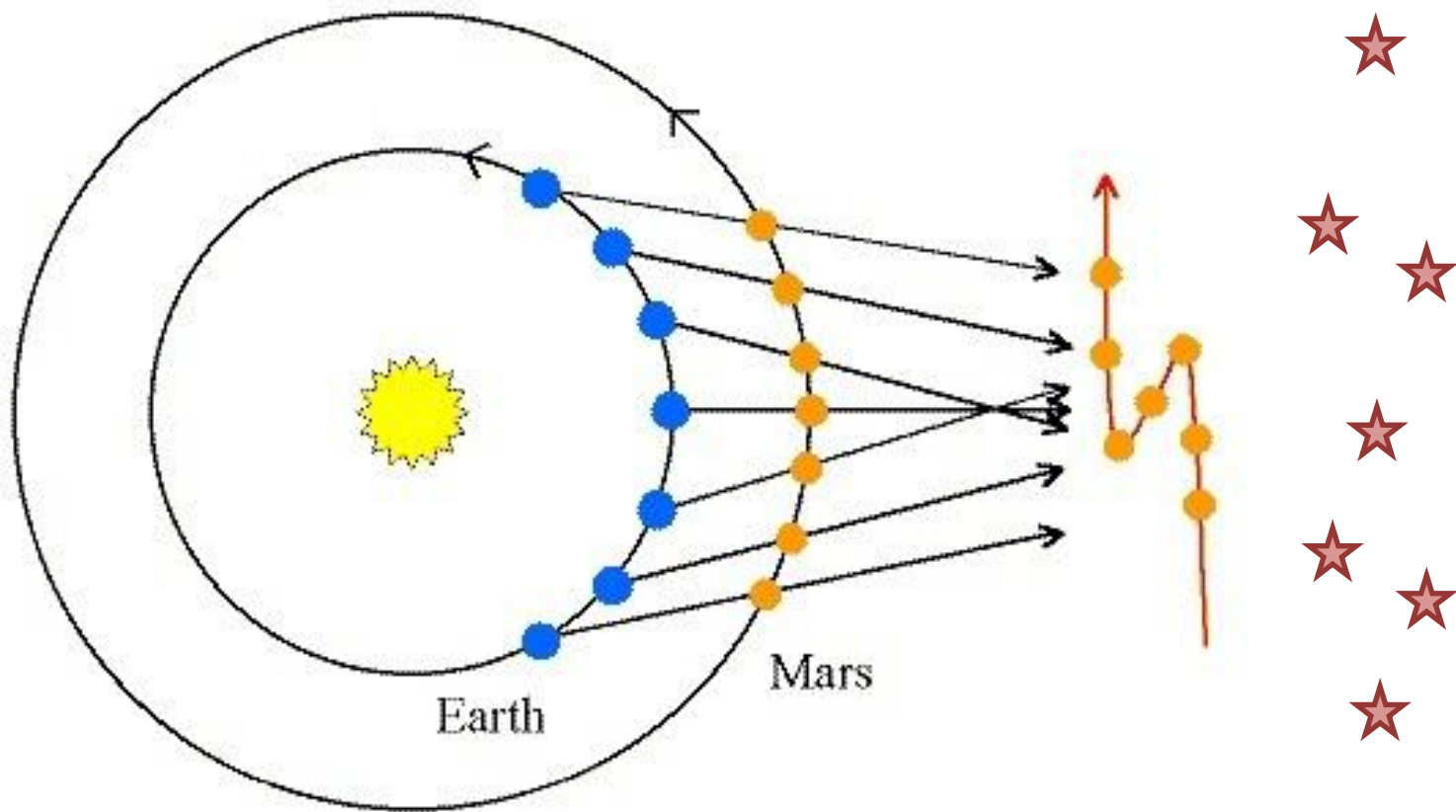


Copernicus

- Nicholas Copernicus (1473-1543) observed that the orbits of the planets can be explained more simply in a heliocentric universe.
- (Images from Wikimedia Commons)







Some Useful Coordinate Systems

- World space
- Object space
- Camera space
- Upright space

World Space

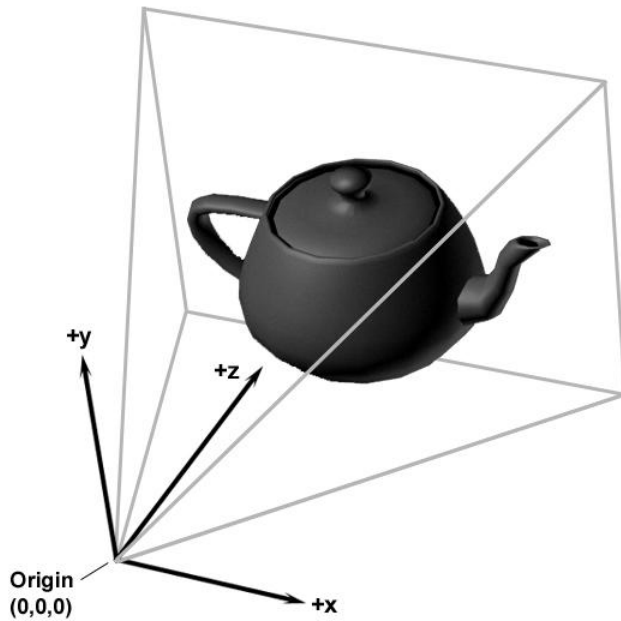
- World space is the global coordinate system.
- Use it to keep track of position and orientation of every object in the game.
- Just like in the movie *Highlander*, there can be only one.

Object Space

Every object in the game has:

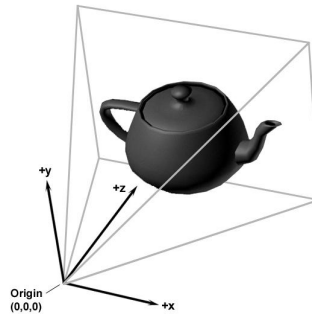
- Its own origin (where it is),
- Its own concept of “up” and “right” and “forwards”,
- That is, its own coordinate space.
- Use it to keep track of relative positions and orientation (eg. Collision detection, AI)

Camera Space



Object space for the viewer, represented by a camera, used to project 3D space onto screen space

Camera Space Terminology

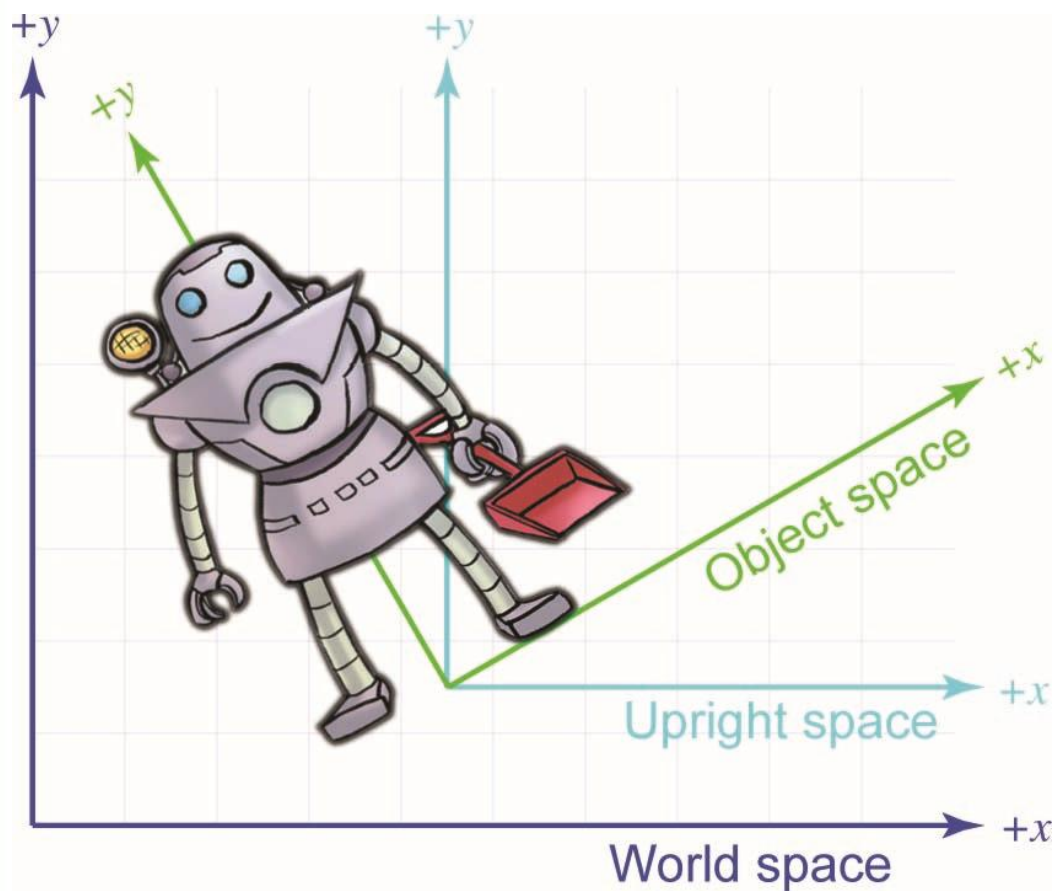


- *Frustum*: the pyramid of space seen by the camera
- *Clipping*: objects partially on screen
- *Occlusion*: objects hidden behind another object
- *Visibility*: inside or outside of frustum, occluded, clipped

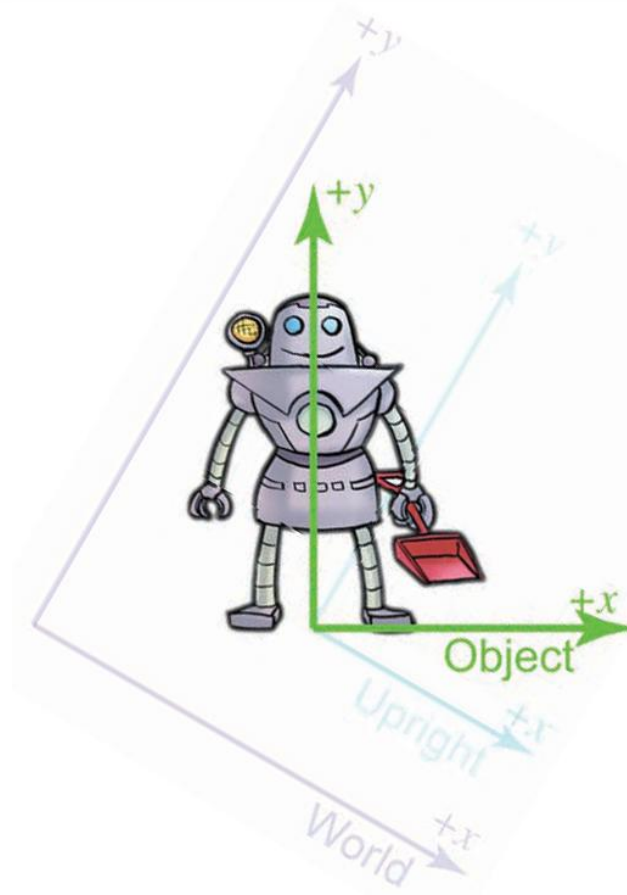
Upright Space

- Upright space is in a sense “half way” between world space and object space.
- Upright space has
 - Object space origin
 - World space axes
- It is nonstandard, we use it in our book because we like it.

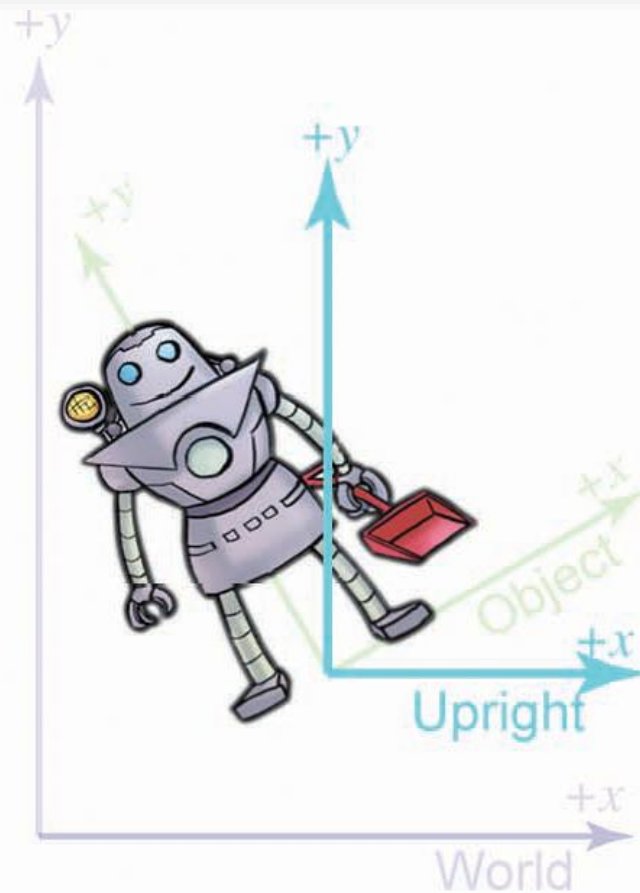
Object, Upright, and World Space



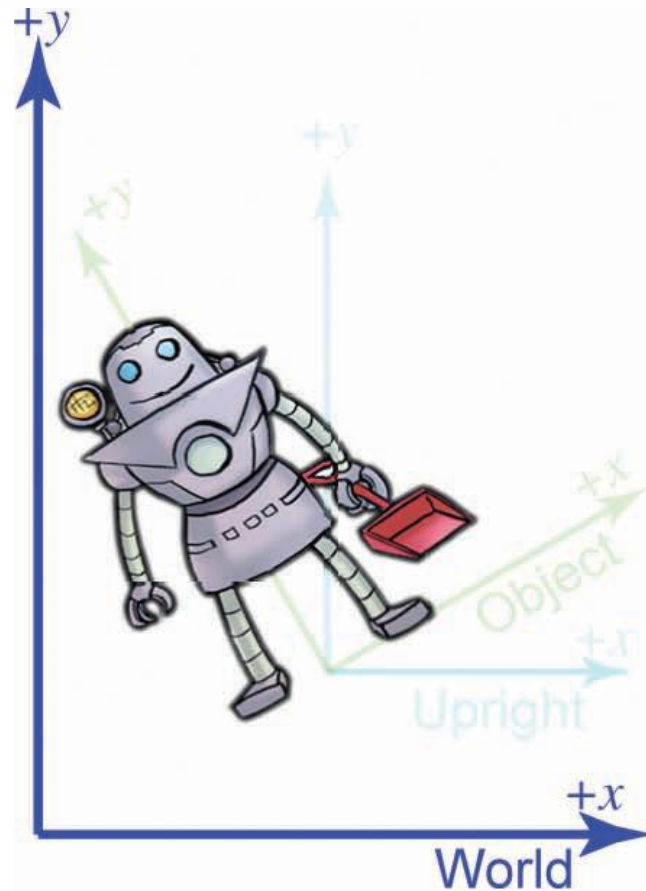
Robot's Object Space



Robot's Upright Space



Robot in World Space



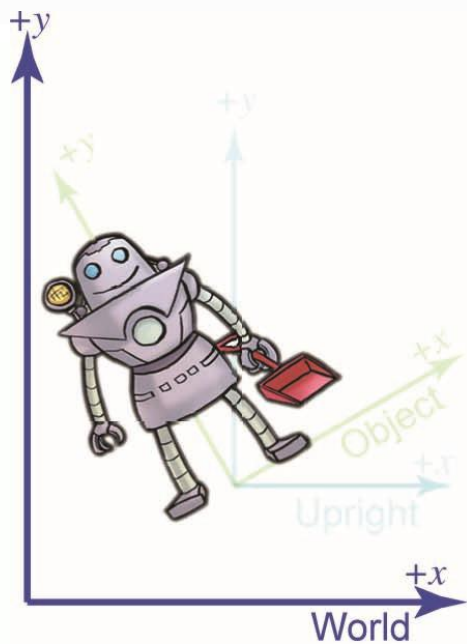
Why Upright Space?

- It separates translation and rotation
 - It is a handy visualization tool
 - It is inspired by both math and hardware implementation
- Translate between world and upright space.
- Rotate between upright and object space.
- Which brings us to *coordinate space transformations...*

Coordinate Space Transformation

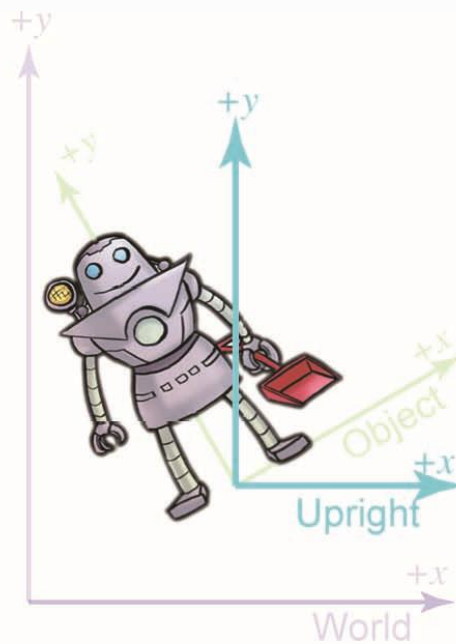
Two important types of transformation used to transform one coordinate space into another:

- Translation
 - Changes position in space
 - Gives new location of origin
- Rotation
 - Changes orientation in space
 - Gives new directions of axes



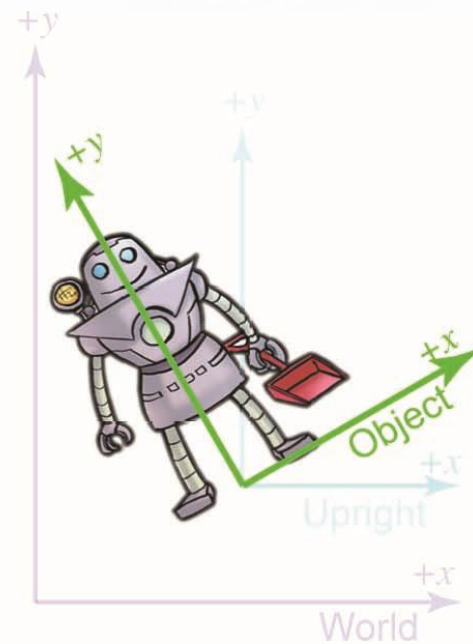
World
Space

Translate
↔



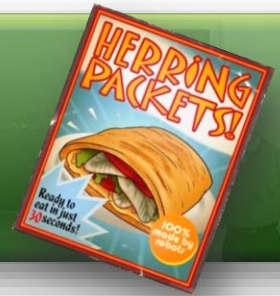
Upright
Space

Rotate
↔



Object
Space

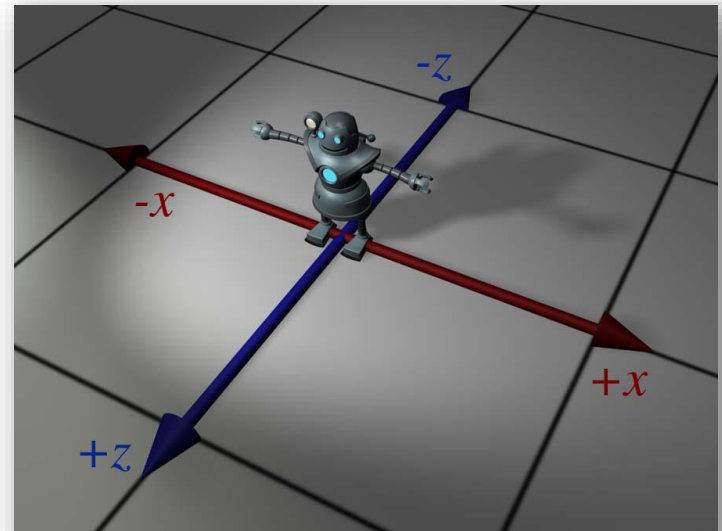
Example

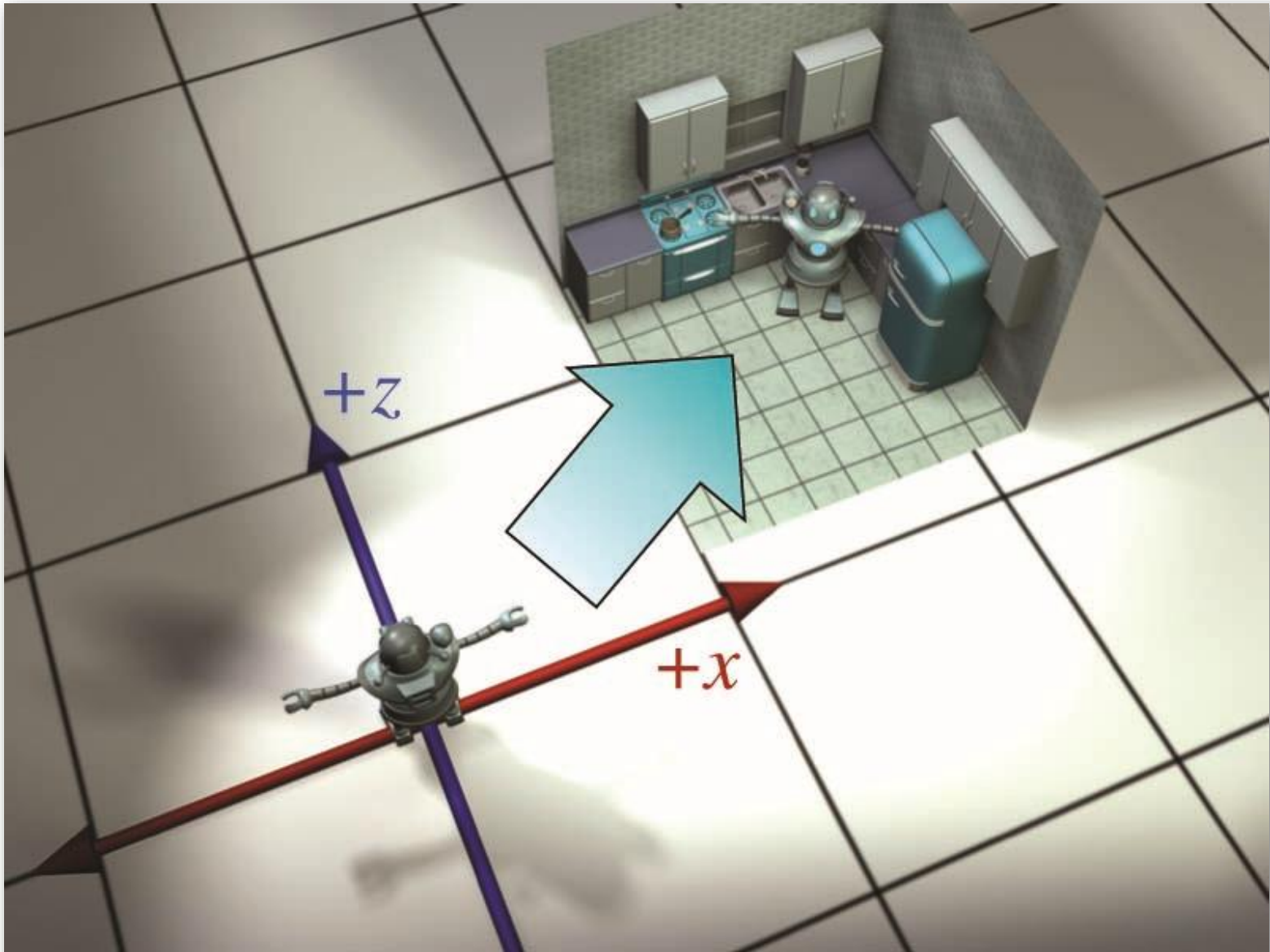


- Let's say that we are working for a advertising agency that has just landed a big account with a food manufacturer.
- You are assigned to the project to make a slick computer-generated ad promoting one of their most popular items, *Herring Packets*, which are microwaveable herring sandwiches food products for robots.
- Of course, the client has a tendency to want changes made at the last minute, so we need to be able to get a model of the product at any possible position and orientation.

Start with the Artist's Model

- For now, because we have the model in its home position, object space and world space (and upright space) are all the same by definition.
- For all practical purposes, in the scene that the artist built containing only the model of the robot, *world space is object space*.

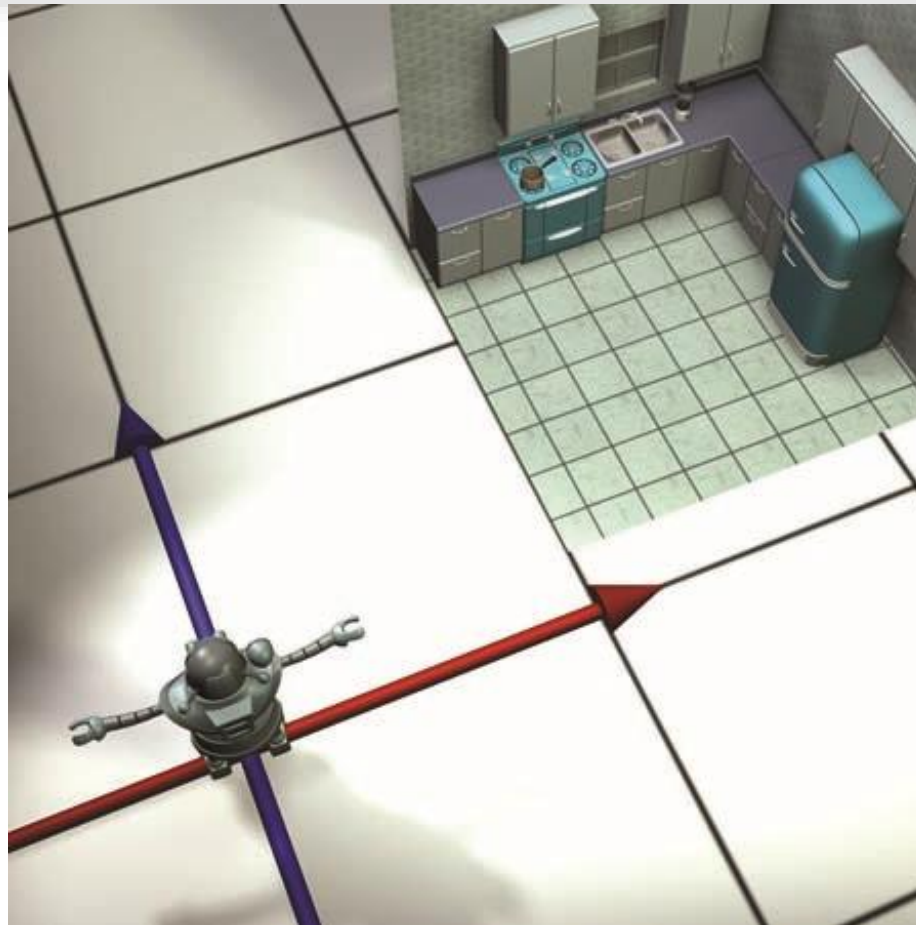




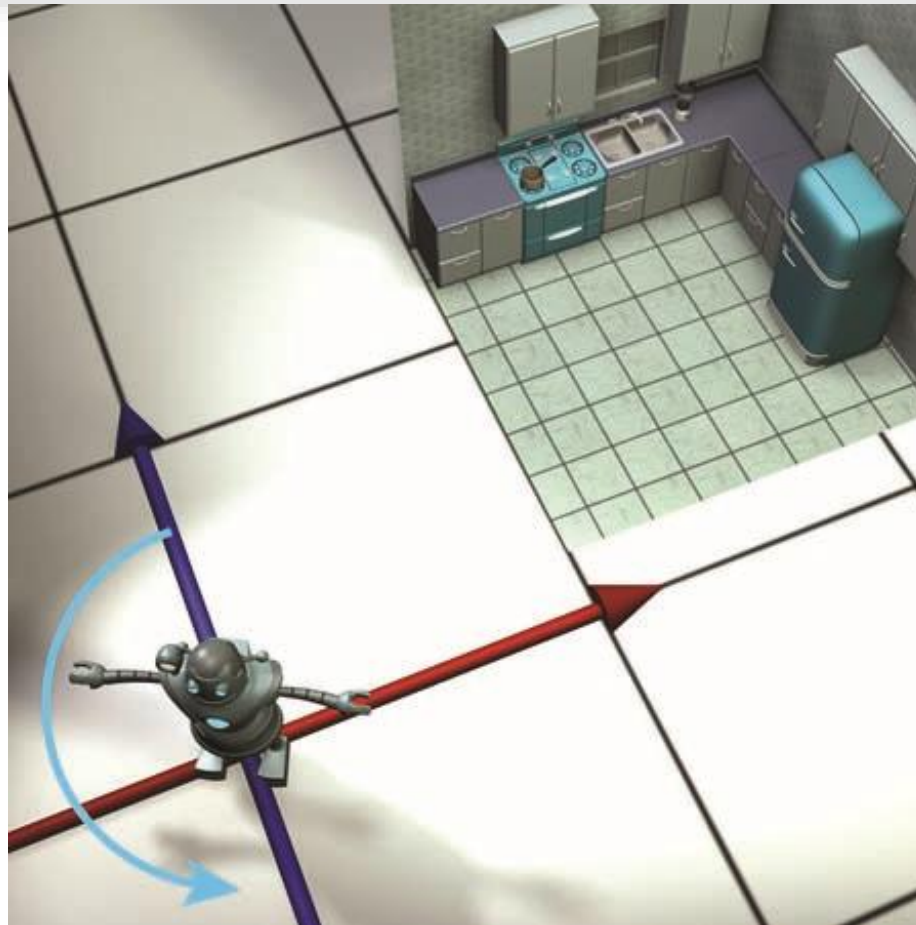
More Details

- Let's talk a bit about how to accomplish this. We won't get too far into the mathematical details - that's what the rest of this chapter is for.
- Conceptually, to move the robot into position we first rotate her clockwise 120° (or, as we'll learn in Chapter 8, by “heading left 120° ”).
- Then we translate 18ft east and 10ft north, which according to our conventions is a 3D displacement of $[18, 0, 10]$.

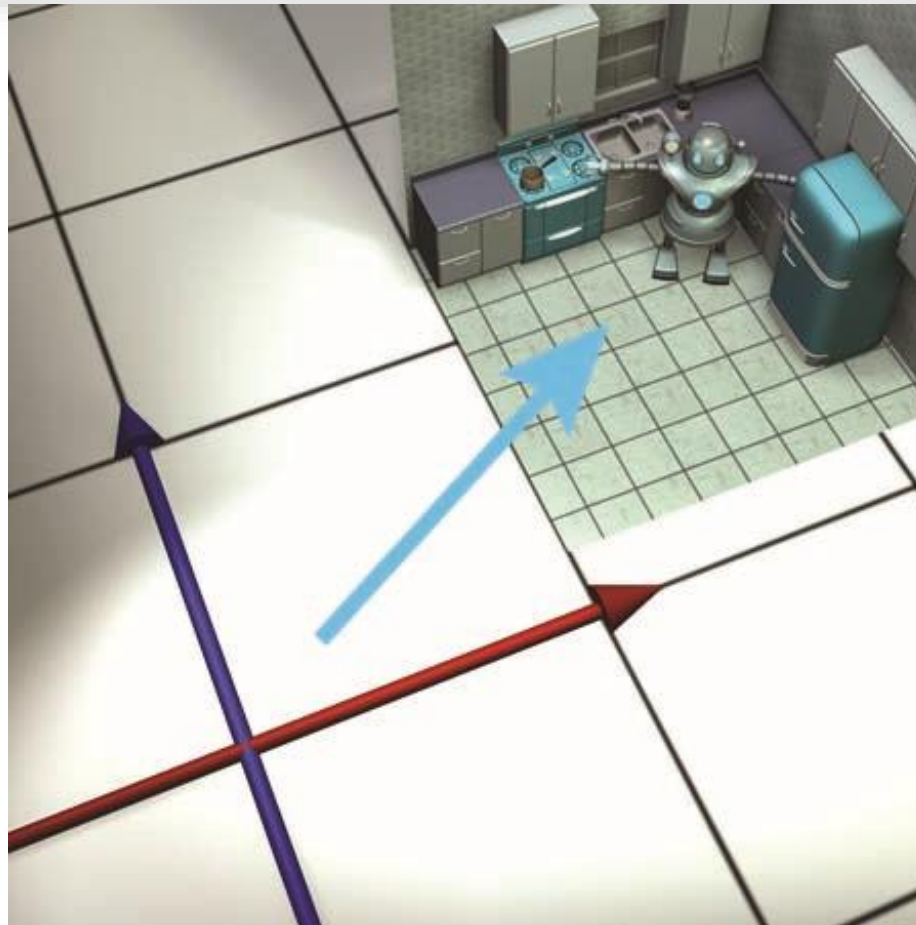
Original Position



Rotate



Then Translate

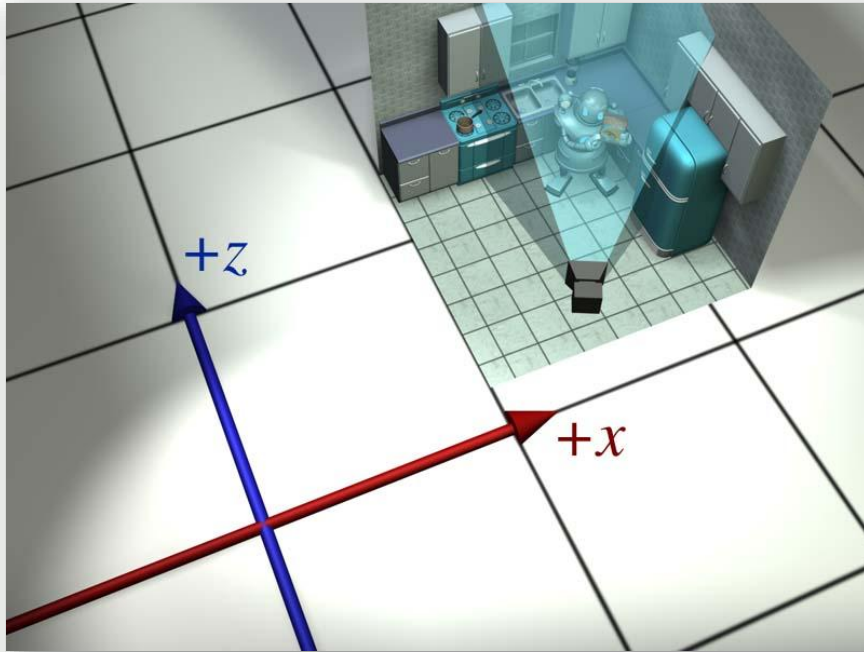


Rotate Before Translate

- Why rotate before we translate?
- Rotation about the origin is easier – translating first would mean that we have to rotate around an arbitrary point.
- Rotation about the origin is a linear transform.
- Rotation about an arbitrary point is an affine transform.
- Affine transforms can be expressed as a sequence of primitive operations.

Camera Space

- So we've managed to get the robot model into the right place in the world.
- But to render it, we need to transform the vertices of the model into *camera space*.
- In other words, we need to express the vertices' coordinates relative to the camera. For example, if a vertex is 9ft in front of the camera and 3ft to the right, then the z and x coordinates of that vertex in camera space would be 9 and 3, respectively.



Where the camera is

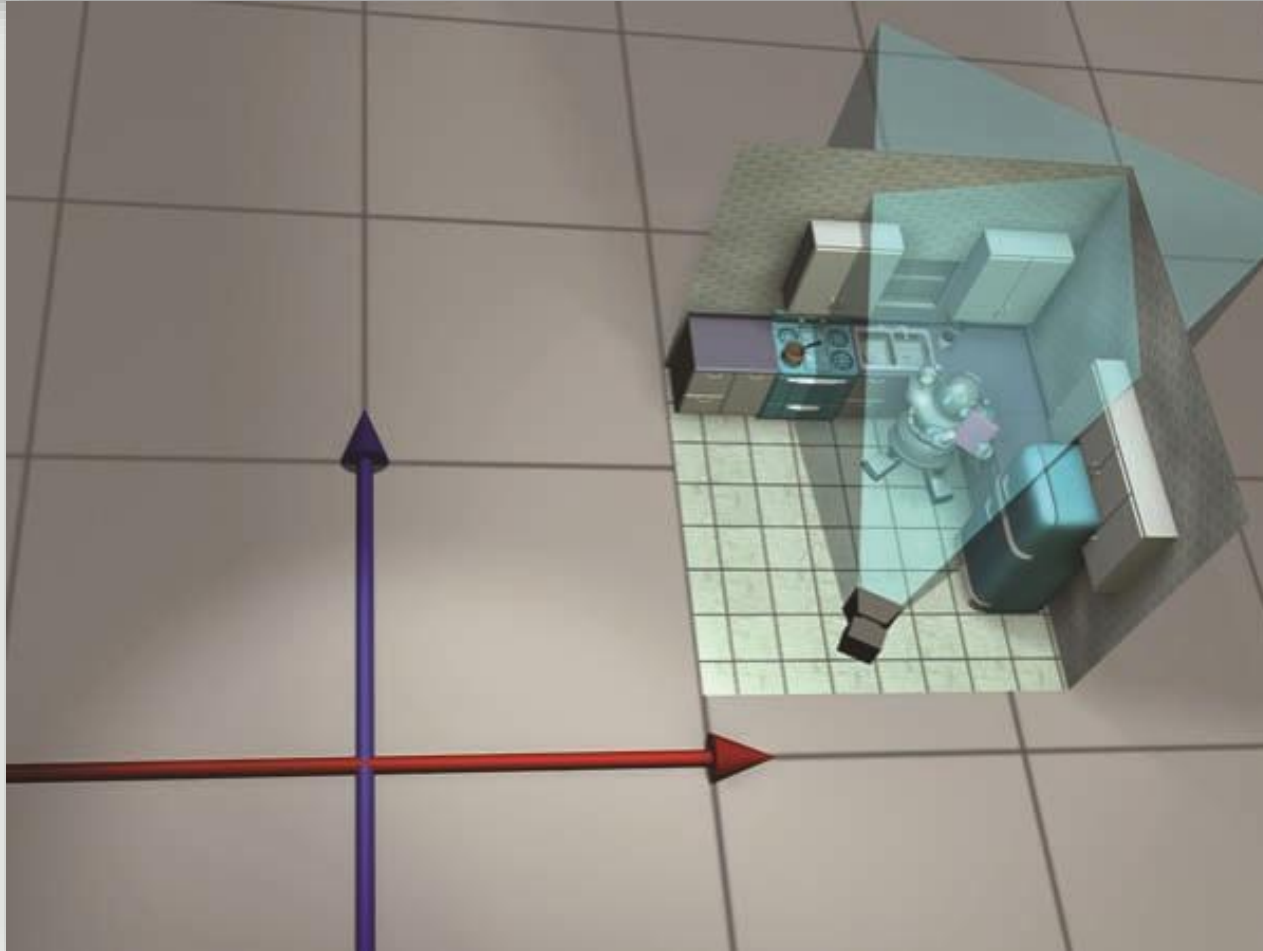


What the camera sees

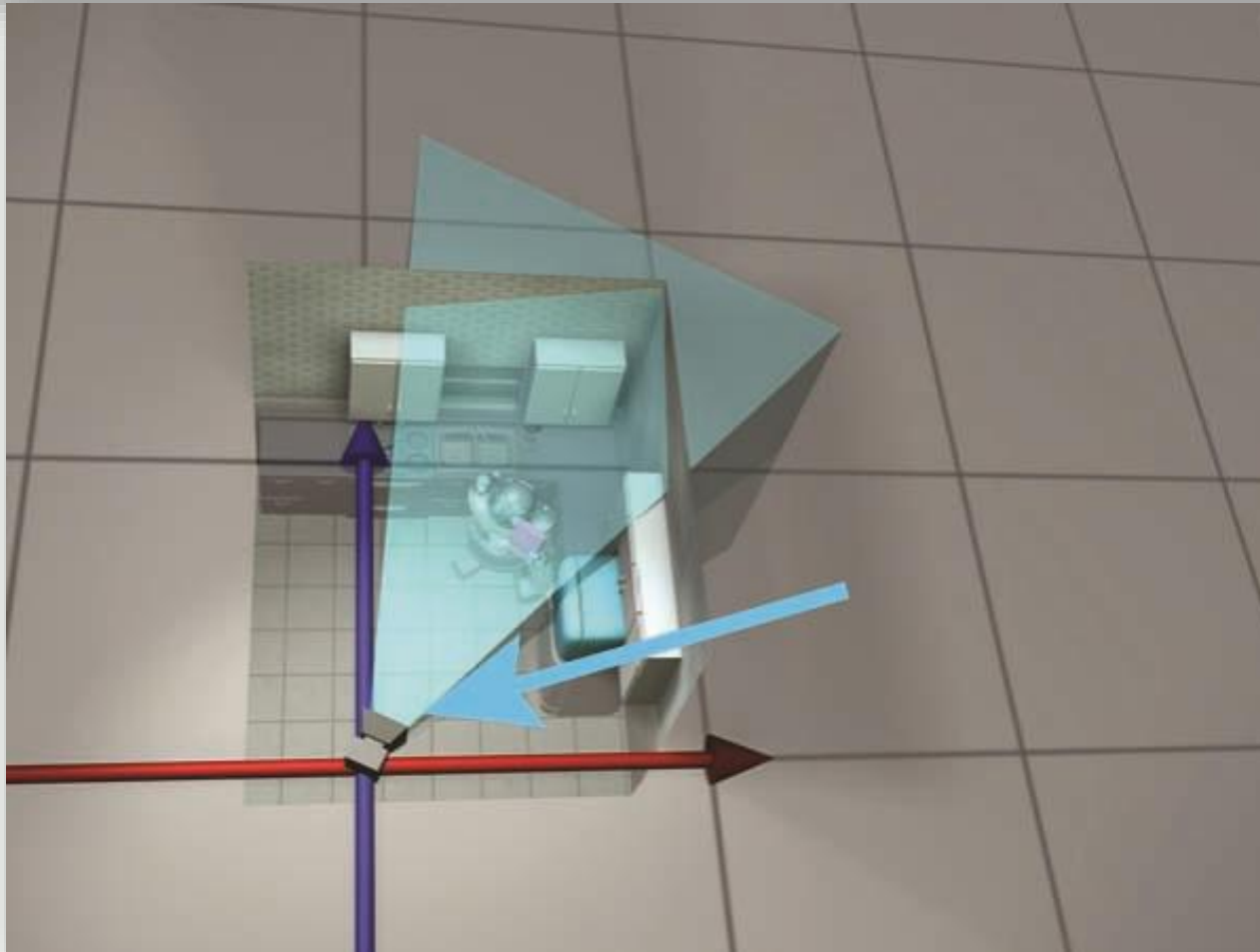
Camera Space

- It's easier to reason about a camera at the origin, looking along a primary axis.
- So we move the whole world so that the camera is at the origin.
- First we translate, then we rotate (for the same reason as before, because rotation about the origin is easier than rotation about an arbitrary point).

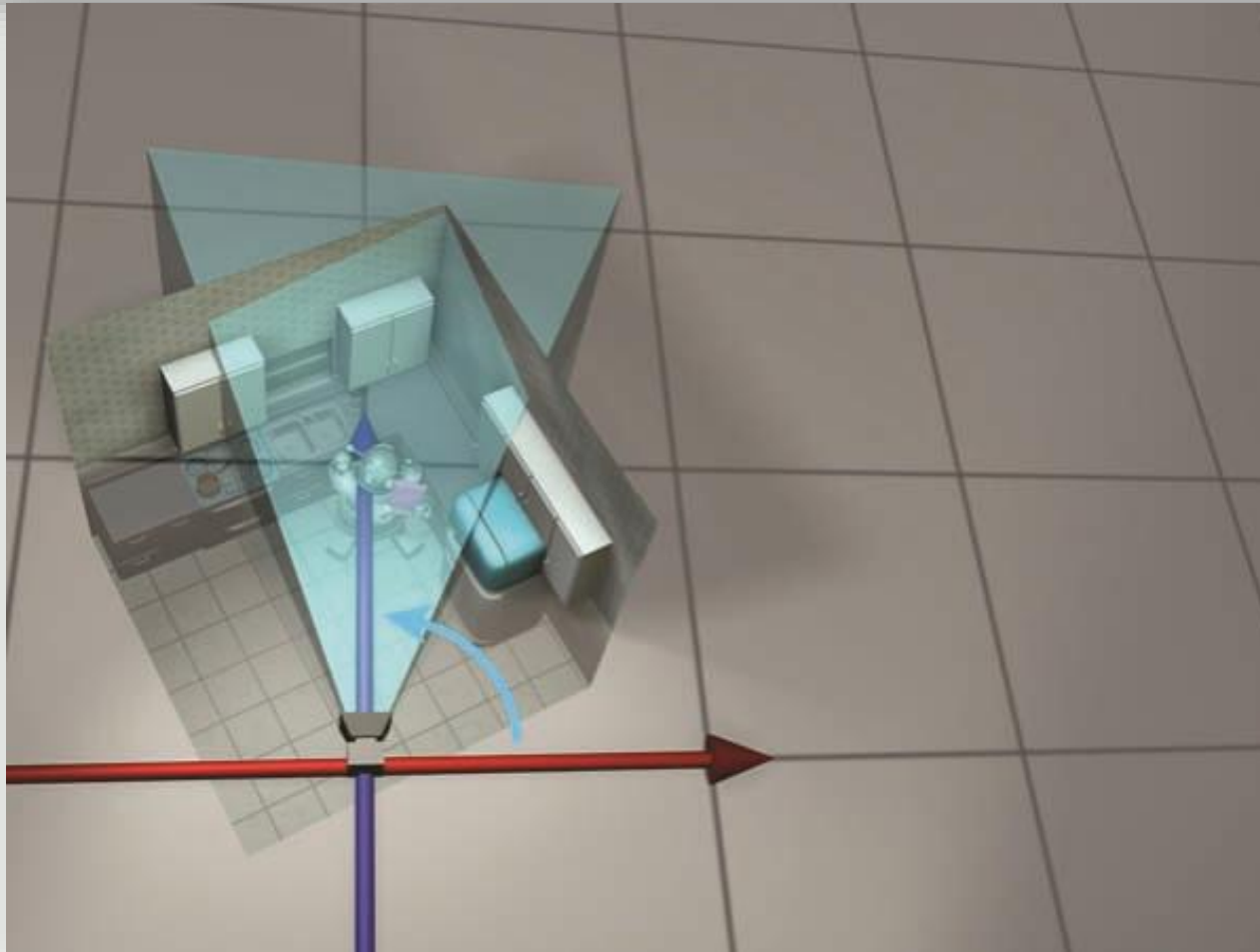
Original Position



Translate the World



Rotate the World



Pipeline Gráfica

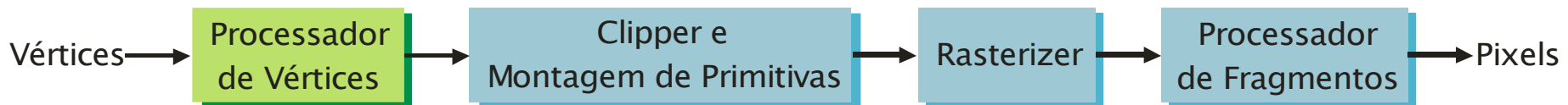
- ▶ Processa objetos um por vez na ordem em que são gerados pela aplicação.
- ▶ Arquitetura



- ▶ Todos os passos podem ser implementados em hardware na placa gráfica.

Processando Vértices

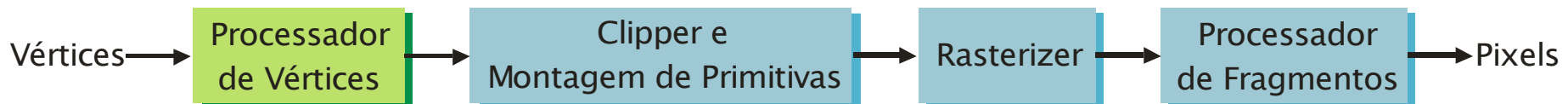
- ▶ A maior parte do trabalho está na conversão da representação de um objeto de um sistema de coordenadas para outro.
 - Coordenadas do objeto
 - Coordenadas do mundo
 - Coordenadas da câmera
 - Coordenadas da janela



- ▶ Processador de vértices também processa cores de vértices.

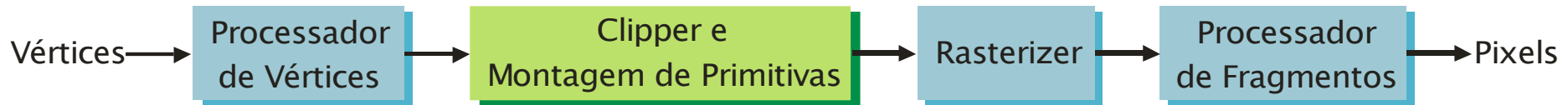
Projeção

- ▶ Processo que processa os objetos 3D para gerar uma imagem 2D correspondente a visão que o observador terá da cena.
 - Projeções de perspectiva: todas as projeções convergem para o centro da projeção.
 - Projeção paralela: centro de projeção é substituído por uma direção de projeção.



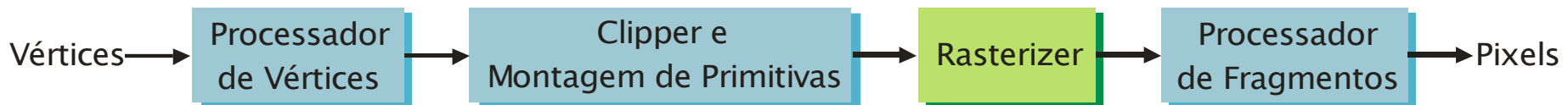
Montagem de Primitivas

- ▶ Vértices devem ser mapeados em objetos geométricos antes da clipagem (clipping), para a rasterização (rastering).
 - Segmentos de linha
 - Polígonos
 - Curvas e superfícies



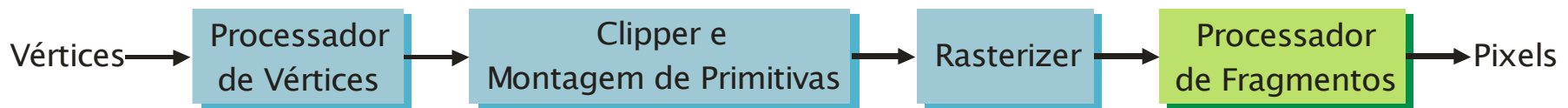
Rasterização

- ▶ Quando um objeto não é clipado, os pixels do frame buffer devem receber cores.
- ▶ A rasterização produz um conjunto de fragmentos para cada objeto.
- ▶ Os fragmentos são “pixels potenciais”
 - Possuem uma localização no frame buffer
 - Cores e atributos de profundidade.

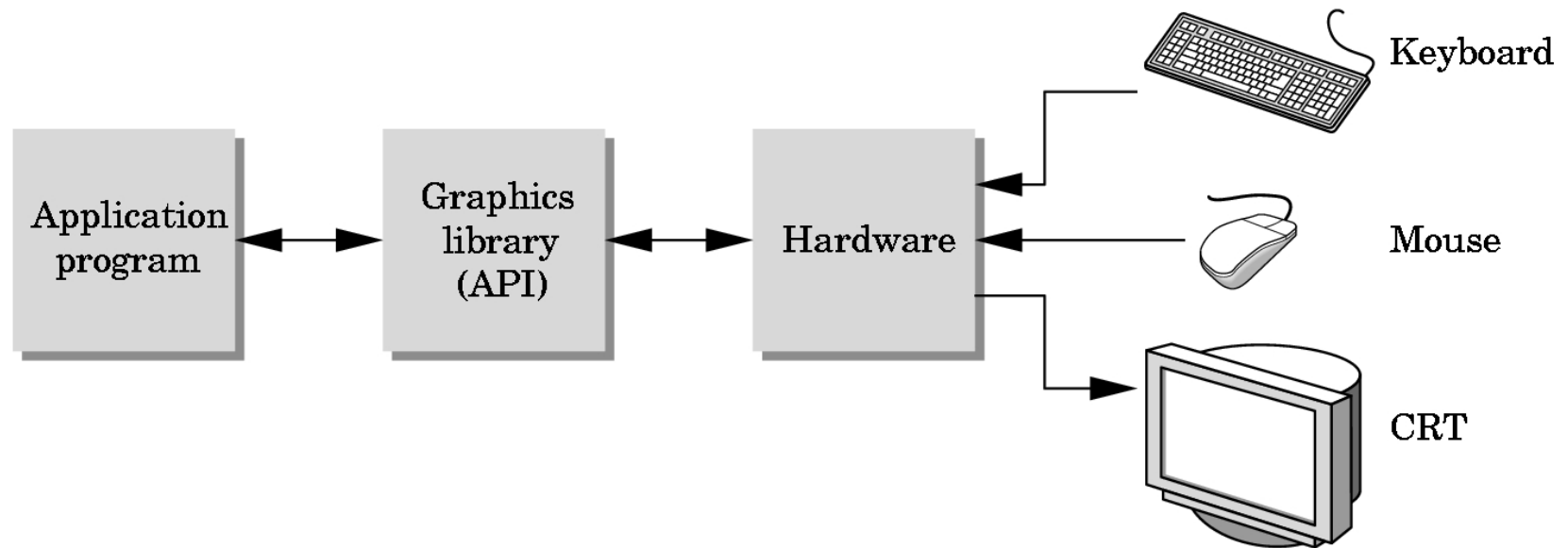


Processando Fragmentos

- ▶ Fragmentos são processados para determinar a cor dos pixels correspondentes.
- ▶ As cores podem ser determinadas por mapeamento de textura ou interpolação das cores dos vértices.
- ▶ Fragmentos podem ser bloqueados por outros fragmentos mais perto da câmera.
 - Remoção de superfície oculta.



Bibliotecas Gráficas



Bibliotecas Gráficas

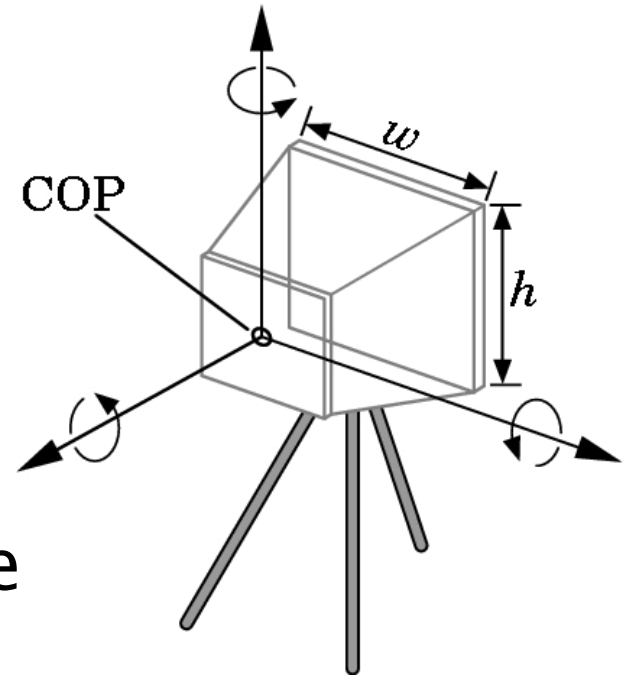
- ▶ Funções que especificam o que precisamos para formar uma imagem
 - Objetos
 - Espectador
 - Luzes de origem
 - Materiais
- ▶ Outras informações
 - Dispositivos como mouse e teclado
 - Capacidades do sistema

Especificação de Objetos

- ▶ A maioria das APIs suportam um conjunto limitado de primitivas incluindo:
 - Pontos (objeto 0D)
 - Segimentos de linha (objetos 1D)
 - Polígonos (objetos 2D)
 - Curvas e superfícies
 - Quadráticas
 - Polinômios paramétricos

Especificação da Câmera

- ▶ Seis graus de liberdade
 - Posição do centro da lente
 - Orientação
- ▶ Lentes
- ▶ Tamanho do filme
- ▶ Orientação do plano do filme



Luzes e Materiais

▶ Tipos de luzes

- Pontuais
- Luzes Spot
- Distância da fonte de luz
- Propriedades de cores

▶ Propriedades dos materiais

- Absorção: propriedades de cor
- Espalhamento
 - Difuso
 - Especular